

Trabajo Práctico Integrador: Estructuras de Datos, Contrato Equals/HashCode y Complejidad Algorítmica

Asignatura: Desarrollo de Software

Modalidad: en grupo

Integrantes del Grupo:

Nombre y Apellido	Amaro Ponce	Sofia Zarate	Facundo Salinas	Francisco Morales	Yaco Santoro	Matías Herrera
Legajo	52082	52167	50166	49123	51084	47697

Criterio de Evaluación y Política de IA: Se permite el uso de herramientas de Inteligencia Artificial (LLMs). Sin embargo, el práctico está diseñado para evaluar el **razonamiento crítico, el análisis de estados de memoria y la justificación técnica**. Las respuestas genéricas generadas por IA sin prueba de trazabilidad manual o sin responder a los escenarios de borde específicos serán penalizadas.

Criterios Generales de Entrega

- Entregar un informe en formato PDF publicado en un repositorio de GitHub.
- Incluir fragmentos de código fuente en Java y los diagramas/trazas explicativas solicitadas.

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Un equipo de desarrollo junior implementó el siguiente sistema para gestionar el acceso de empleados a un edificio mediante tarjetas de identificación mutable. El código no lanza excepciones en tiempo de ejecución, pero presenta comportamientos anómalos en producción.

```

import java.util.*;
class TarjetaAcceso {
private String codigo;
private int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
this.codigo = codigo;
    this.nivelAcceso = nivelAcceso;
    }
    public void setNivelAcceso(int nivelAcceso) {
this.nivelAcceso = nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
TarjetaAcceso tarjeta = (TarjetaAcceso) o;
    return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo,
tarjeta.codigo);
    }

    @Override
    public int hashCode() {
    return Objects.hash(codigo, nivelAcceso);
    }
}

public class ControlAcceso {

    public static void main(String[] args) {
        Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();[cite: 3]
TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);

        tarjetasValidas.add(t1);

        // Modificación posterior a la inserción          t1.setNivelAcceso(5);

        System.out.println("¿Tarjeta válida en sistema?: " +
tarjetasValidas.contains(t1));
        System.out.println("Cantidad de tarjetas registradas: " +
tarjetasValidas.size());
    } }

```

Responder y Justificar:

1. **Traza del Estado Interno:** Explique paso a paso qué ocurre dentro de la estructura HashSet:
 - Al ejecutar tarjetasValidas.add(t1). o
 - Al ejecutar t1.setNivelAcceso(5).
 - Al ejecutar tarjetasValidas.contains(t1).

2. **Explicación del Cajón (Bucket):** ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer `contains()`?
3. **Fuga de Memoria (*Memory Leak*):** ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?
4. **Refactorización Obligatoria:** Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

1. a) Al ejecutar `tarjetasValidas.add(t1)`, el objeto `t1` se agrega al `HashSet`. Para ubicarlo dentro de la estructura, `HashSet` utiliza el valor obtenido por `hashCode()`. Como `hashCode()` depende de `codigo` y `nivelAcceso`, en ese momento se calcula usando los valores "ACC-101" y 2. Con ese hash, el objeto queda almacenado en el bucket correspondiente. Si ya existiera algún elemento candidato en ese bucket, `equals()` permitiría comprobar si es lógicamente igual antes de agregarlo.

b) Al ejecutar `t1.setNivelAcceso(5)`, se modifica el atributo `nivelAcceso` del mismo objeto `t1`, pasando de 2 a 5. Como `nivelAcceso` participa en el cálculo de `hashCode()`, el hash que produciría el objeto a partir de ese momento puede ser diferente al que tenía cuando fue agregado al `HashSet`.

Sin embargo, el `HashSet` no reubica automáticamente el objeto cuando cambia su estado, por lo que `t1` permanece almacenado en el bucket asociado al hash anterior.

c) Cuando se ejecuta `tarjetasValidas.contains(t1)`, el `HashSet` vuelve a calcular el `hashCode()` del objeto para determinar en qué bucket debe buscarlo. Como `nivelAcceso` ahora vale 5, el hash calculado puede ser distinto del que tenía cuando se insertó.

Por este motivo, el `HashSet` puede buscar en un bucket diferente, mientras que `t1` continúa almacenado en el bucket correspondiente al hash anterior. Como no encuentra el objeto en el bucket donde lo está buscando, `contains(t1)` puede devolver `false`, aunque el objeto continúe dentro del conjunto.

Por eso `tarjetasValidas.size()` sigue devolviendo 1: el elemento nunca fue eliminado, simplemente quedó almacenado en una ubicación que ya no coincide con su hash actual.

2. al agregar `t1` al `HashSet`, su `hashCode()` se calcula usando los valores actuales de `codigo` y `nivelAcceso`. En la ejecución realizada, el hash inicial fue -1387020803, y con ese valor la estructura ubicó el elemento internamente.

Luego se modificó `nivelAcceso` de 2 a 5. Como este atributo participa en `hashCode()`, el nuevo hash pasó a ser -1387020800. Sin embargo, el `HashSet` no reubica automáticamente el objeto cuando cambia su estado, por lo que este permanece almacenado según el hash anterior.

Cuando se ejecuta `contains(t1)`, se calcula nuevamente el `hashCode()` actual y se busca utilizando el nuevo valor. Como el objeto sigue almacenado según el hash viejo, la búsqueda puede fallar y devolver `false`, aunque `size()` siga devolviendo 1 porque el objeto continúa dentro del conjunto.

3. Se produce una fuga lógica de memoria porque el objeto `t1` continúa almacenado dentro del `HashSet`, como demuestra que `size()` sigue devolviendo 1, pero al haberse modificado un atributo que participa en `hashCode()`, la estructura puede dejar de encontrarlo mediante `contains()` o `remove()`. El objeto ocupa espacio dentro de la colección, pero queda inaccesible mediante el mecanismo normal de búsqueda basado en hash.

```

tarjetasValidas.add(t1);

System.out.println("Hash antes: " + t1.hashCode());

t1.setNivelAcceso(5);

System.out.println("Hash despues: " + t1.hashCode());

```

```

\tp-estructuras-datps> java ControlAcceso
Hash antes: -1387020803
Hash despues: -1387020800
Tarjeta valida en sistema?: false
Cantidad de tarjetas registradas: 1
PS C:\Users\Usuario\Desktop\Desarrollo de Software 2026\Unidad 5 - ToString, Equals y hashCode. Coleccion
\tp-estructuras-datps>

```

4. 2 Soluciones osibles para corregir el problema son:

- **Hacer la clase inmutable.** Hacer que los atributos que participan en `equals()` y `hashCode()` sean finales

```

private final String codigo;
private final int nivelAcceso;

public TarjetaAcceso(String codigo, int nivelAcceso) {
    this.codigo = codigo;
    this.nivelAcceso = nivelAcceso;
}

public void setNivelAcceso(int nivelAcceso) {
    this.nivelAcceso = nivelAcceso;
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;

    TarjetaAcceso tarjeta = (TarjetaAcceso) o;

    return nivelAcceso == tarjeta.nivelAcceso
        && Objects.equals(codigo, tarjeta.codigo);
}

@Override
public int hashCode() {
    return Objects.hash(codigo, nivelAcceso);
}
}

```

Con esto no podemos hacer `t1.setNivelAcceso(*),` ya que no se pueden modificar los atributos

- **Cambiar el criterio de identidad.** Hacer que el atributo `nivelAcceso` no participe en la identidad lógica. Que solo se use el atributo código.

```

public class TarjetaAcceso {

    private String codigo;
    private int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;

        TarjetaAcceso tarjeta = (TarjetaAcceso) o;

        return
            Objects.equals(codigo, tarjeta.codigo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(codigo);
    }
}

```

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Diseñe el backend de un módulo para una plataforma de *E-Commerce*. Debe elegir la interfaz y la implementación concreta del Java Collections Framework (`java.util`) más eficiente para cada una de las siguientes tres funcionalidades:

Requerimiento 1: Sistema de Auditoría de Transacciones

Registrar operaciones en el orden exacto en que ocurren. Se requiere insertar miles de registros por segundo al final de la colección y recorrerlos secuencialmente.

Requerimiento 2: Catálogo de Ofertas Relámpago

Mantener los productos ordenados automáticamente por precio de menor a mayor. Debe permitir consultar eficientemente el producto más barato y el más caro.

Requerimiento 3: Búsqueda de Sesiones de Usuarios Activos

Buscar instantáneamente la sesión de un usuario dado su Token (String único) entre 500,000 usuarios conectados en simultáneo.

Responder y Justificar:

Completar la siguiente tabla comparativa y justificar la decisión con la complejidad algorítmica Big O:

Requerimiento	Interfaz Seleccionada (List , Set , Map , Queue) PDF	Implementación Concreta (ArrayList , TreeSet , HashMap , etc.) PDF	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
1. Auditoría					
2. Ofertas					
3. Sesiones					

Requerimiento	Interfaz	Implementación	Complejidad Búsqueda	Complejidad inserción	Justificación
1. A	List	ArrayList	O(n)	O(1)	J.A
2. O	Set	TreeSet	O(log n)	O(log n)	J.O
3. S	Map	HashMap	O(1)	O(1)	J.S

J.A: Elegimos la interfaz List porque el requerimiento necesita conservar el orden exacto en que se registran las transacciones y recorrerlas secuencialmente. Como implementación elegimos ArrayList, ya que las inserciones se realizan al final de la colección, operación que tiene una complejidad O(1). La búsqueda de un elemento concreto es O(n), ya que puede ser necesario recorrer la lista.

Aunque una inserción en una posición intermedia sería O(n), debido al desplazamiento de los elementos posteriores.

```
List<Transaccion> auditoria = new ArrayList<>();
```

J.O: Elegimos la interfaz Set y como implementación TreeSet porque necesitamos los productos ordenados automáticamente por precio. TreeSet utiliza un criterio de orden mediante Comparable o Comparator, y sus operaciones de búsqueda e inserción tienen complejidad O(log n), ya que no necesitan ver todos los elementos, pero tampoco pueden poner por poner, tiene que ir como "recorriendo un árbol"

```
Set<Producto> ofertas = new TreeSet<>(porPrecio);
```

J.S: Elegimos la interfaz Map porque necesitamos asociar un token único con la sesión correspondiente (Token → Sesión). Como implementación utilizamos HashMap porque el requerimiento no necesita mantener ningún orden y prioriza encontrar rápidamente una sesión entre una gran cantidad de usuarios. HashMap utiliza el `hashCode()` de la clave para localizar el bucket

correspondiente y `equals()` para distinguir claves cuando es necesario. Por este motivo, tanto la búsqueda mediante `get()` como la inserción mediante `put()` tienen complejidad promedio $O(1)$.

```
Map<String, Sesion> sesiones = new HashMap<>();
```

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

Un desarrollador escribió el siguiente método para encontrar el primer carácter que no se repite dentro de una cadena de texto (ejemplo: en "barco", el primero es 'b'; en "abarca", el primero es 'r').

```
public static Character primerCaracterNoRepetidoLento(String texto) {
    // Algoritmo ineficiente  $O(n^2)$ 
    for (int i = 0; i < texto.length(); i++)
    {
        char c = texto.charAt(i);
        boolean duplicado = false;
        for (int j = 0; j < texto.length(); j++) {
            if (i != j && c == texto.charAt(j)) {
                duplicado = true;
                break;
            }
        }
        if (!duplicado) return c;
    }
    return null;
}
```

Responder y Justificar:

1. **Refactorización a $O(n)$:** Escriba una versión optimizada del método utilizando la colección adecuada de `java.util` (`HashMap`, `LinkedHashMap`, o `HashSet`) que resuelva el problema realizando como máximo dos recorridos lineales ($O(n)$).
2. **Análisis de Colisiones:** Si se utiliza un `HashMap` internamente, explique qué ocurre si dos caracteres distintos generan el mismo `hashCode()` (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

1. Reemplazamos el algoritmo ineficiente por:

```
// Aca vamos a guardar la cantidad de veces que aparece una letra
```

```
Map<Character, Integer> frecuencias = new HashMap<>();
```

```
// Primer recorrido: acá vamos a contar cuantas veces aparece cada letra
```

```
for (int i = 0; i < texto.length(); i++) {
```

```
    char c = texto.charAt(i);
```

```
    if (frecuencias.containsKey(c)) {
```

```
        frecuencias.put(c, frecuencias.get(c) + 1);
```

```
    } else {
```

```
frecuencias.put(c, 1);
```

```
}}
```

// Segundo recorrido: acá vamos a encontrar la primera letra que aparece una sola vez

```
for (int i = 0; i < texto.length(); i++) {
```

```
    char c = texto.charAt(i);
```

```
    if (frecuencias.get(c) == 1)
```

```
    { return c;
```

```
}}
```

```
return null; }
```

2. Lo que sucede cuando dos caracteres generan el mismo hashCode() es que se genera una colisión, pero esto lo resuelve la estructura, utilizando el equals() para distinguirlas. Por ende, dos elementos con el mismo hashCode() pueden coexistir dentro del mapa

Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String:

```
□ String s1 = "FB"; □  
  String s2 = "Ea";
```

Sabiendo que s1.hashCode() == s2.hashCode() devuelve true (ambos dan el entero 2236):

1. **Si guardamos ambas cadenas en un HashSet<String>, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos hashCode() y equals() internamente para determinar si s2 debe ingresarse o desecharse.**

La colección va a tener 2 elementos.

Ya que el hashCode por sí solo no puede decidir si dos objetos son iguales, solo le indica a Java en cual bucket buscar. Cuando metemos s1 = "FB", Java calcula su hash (2236) y lo ubica en el bucket que corresponde a ese número. Cuando metemos s2 = "Ea", como también da 2236, Java va a revisar ese mismo bucket antes de insertarlo/depositarlo.

Ahí entra equals(): como ya hay algo en ese bucket, Java compara s1.equals(s2) para chequear si el contenido es realmente el mismo. Como "FB" y "Ea" son cadenas distintas, equals() devuelve false, entonces Java entiende que no es un duplicado y agrega a s2 como un elemento nuevo dentro de ese mismo bucket (conviven ahí adentro, en una lista o estructura interna).

Entonces podemos decir que: coincidir en el hash no alcanza para que Java los trate como iguales, la decisión final siempre pasa por equals().

2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un HashMap en el peor escenario posible (de $O(1)$ a $O(n)$ o $O(\sqrt{\log n})$)?

En un caso ideal, cada clave distinta cae en un bucket distinto, entonces `get()` y `put()` son casi instantáneos: **$O(1)$** .

Pero hay un problema, que aparece cuando muchas claves distintas colisionan y terminan amontonadas en el mismo bucket (como pasaría acumulando muchos casos tipo "FB"/"Ea"). Ahí, para encontrar la clave correcta, Java tiene que recorrer los elementos de ese bucket comparando con `equals()` uno por uno, como si fuera una lista. En el peor escenario posible, con todas las claves en el mismo bucket, la complejidad se degrada a **$O(n)$** .

Como dato extra: desde Java 8, si un bucket acumula más de 8 elementos, Java lo convierte internamente en un árbol balanceado en vez de dejarlo como lista, así que en la práctica ese peor caso queda en **$O(\log n)$** en vez de $O(n)$ puro.